

Backlog - Sprint 3 (ABP Sistema de Leads)

Objetivo da Sprint 3

Implementar funcionalidades avançadas de gestão de leads e negociações, finalizar o sistema de logs e auditoria, e entregar a documentação completa do projeto.

Correção Prioritária (Feedback da Sprint 2)

Endereçar todos os pontos levantados no relatório de inspeção antes de avançar novas funcionalidades.

- Preencher docs/retrospectivas/sprint-2.md com conteúdo real de retrospectiva, revisão, ações de melhoria e evidências de aprendizado
- Corrigir README: versão do React (19.2.4), paths de dashboard (/api/dashboard e /api/dashboard/analytics) e afirmação sobre seed automático no Docker
- LoginService: substituir Error comum por AppError com status 401 em credenciais inválidas (evitar retorno 500)
- Remover JWT_SECRET e senha do PostgreSQL do docker-compose.yml versionado; mover para .env não rastreado
- RBAC: remover perfil LIDER_EQUIPE não previsto no desafio ou justificá-lo na documentação
- RBAC: corrigir filtro de leads do Gerente para escopo de equipe (não storeId)
- RBAC: restringir Gerente Geral a visualização e dashboard — remover permissões de criação/atualização de equipes e exclusão de clientes
- RBAC: adicionar validação de escopo em findById, update e delete de leads (verificar pertencimento ao atendente/equipe autorizado)
- Montar PublicLeadController nas rotas e substituir mockLeadStorage por persistência real no banco
- Kanban: ao avançar estágio, criar/atualizar negociação vinculada e registrar NegotiationHistory; corrigir envio de closingReason obrigatório no fechamento
- Filtros temporais (backend): implementar filtro por semana, mês e ano; validar limite de 1 ano para não-administradores; validar datas inválidas no DashboardController
- Corrigir fórmula de taxa de conversão: leads convertidos ÷ total de leads finalizados; corrigir métrica de não convertidos (excluir leads em andamento)
- Aplicar componente PrivateRoute nas rotas /dashboard, /leads e /perfil no frontend
- Dashboard: consumir endpoints /api/dashboard e /api/dashboard/analytics em vez de calcular métricas no cliente; remover mock do ADMIN

- docs/api.md: documentar parâmetros temporais e modos semana/mês/ano/período customizado; alinhar paths com README
- Docker: implementar seed automático no container do backend ou corrigir README para refletir a realidade
- Eliminar árvore src/resource ou concluir migração; remover importações cruzadas entre nova arquitetura e código antigo
- Corrigir 3 falhas de testes: leadsAdapter.test.ts e useKanbanBoard.test.ts
- Alinhar estados do State Pattern documentados (NOVO, EM_CONTATO...) com estados reais do código (novo_lead, contato...)
- UserService.update: garantir hash explícito de senha na rota de atualização de usuário

Integração Frontend ↔ Backend

- Integrar listagem de leads com endpoint GET /leads (com paginação e filtros)
- Integrar edição de lead com endpoint PUT /leads/:id
- Integrar atualização de status/estágio do lead com a API
- Implementar tratamento global de erros nas requisições (401, 403, 404, 500)
- Armazenar e renovar token JWT no frontend (localStorage / refresh token)

Gestão de Negociações (US08, US09, US10)

Implementar o fluxo completo de negociações vinculadas aos leads, cobrindo criação, atualização de status/estágio e registro de histórico.

- Criar endpoint POST /negotiations e tela de criação de negociação vinculada ao lead (US08)
- Garantir regra de negócio: apenas uma negociação ativa por lead (US08)
- Implementar alteração de status da negociação (aberta/fechada) no frontend e backend (US09)
- Implementar funil de vendas: atualização de estágio (Prospecção → Qualificação → Proposta → Fechamento) (US09)
- Registrar automaticamente no histórico cada mudança de status e estágio com data e responsável (US10)
- Criar tela/componente de visualização do histórico de negociação (US10)
- Criar modal de confirmação ao avançar estágio do lead no Kanban
- Implementar menu de ações (três pontinhos) no card do lead para edição de estágio/status
- Implementar seleção de importância (frio, morno, quente) na criação e edição de negociação, com indicador visual no card

Dashboard Analítico (US11, US12, US13)

- Exibir total de leads com dados reais vindos da API (US11)
- Exibir leads agrupados por status e por origem no dashboard (US11)
- Implementar filtro por período no dashboard (semana, mês, ano, customizado) (US11 + US13)
- Implementar gráfico de taxa de conversão de leads — fórmula: leads convertidos ÷ total de leads finalizados (US12)
- Exibir motivos de finalização de negociações (ganhas/perdidas) (US12)
- Validar filtros de período no backend: limite de 1 ano para não-administradores (US13)
- Buscar e exibir dados adicionais: leads por loja, por equipe, por atendente, tempo médio até atendimento e distribuição por importância

Gestão de Usuários e Equipes (US03, US04)

Finalizar o módulo administrativo de gerenciamento de usuários e equipes, com interface completa no frontend.

- Criar tela de gerenciamento de usuários (CRUD completo) integrada ao backend (US03)
- Implementar definição de papel do usuário (Atendente, Gerente, Gerente Geral, Admin) na interface (US03)
- Criar tela de gerenciamento de equipes com criação, edição e exclusão (US04)
- Implementar associação de usuários às equipes na interface (US04)
- Garantir que rotas do frontend respeitam o papel do usuário (RBAC via PrivateRoute) (US02)
- Atualizar funcionalidades da Sidebar: exibir itens de menu condicionalmente por perfil (Atendente, Gerente, Gerente Geral, Admin)

Logs e Auditoria (US14, US15)

Implementar o sistema de registro e visualização de logs para auditoria e rastreabilidade das ações no sistema.

- Registrar automaticamente eventos de login e logout no banco (US14)
- Registrar operações CRUD em leads, negociações, usuários e equipes (US14)
- Armazenar data, hora e usuário responsável em cada log (US14)
- Criar endpoint GET /logs com filtros por usuário, tipo de ação e período (US15)
- Criar tela de visualização de logs acessível apenas ao Admin (US15)
- Exibir registro de ações no frontend para Gerente (logs da própria equipe) e Admin (logs completos), com filtros de data e tipo de ação

Tela de Gestão de Leads – Kanban (US17)

- Permitir arrastar e soltar cards entre etapas (drag and drop) com atualização na API (US17)
- Exibir informações resumidas do lead no card (nome, atendente, status, importância) (US17)
- Exibir modal de confirmação ao avançar lead para próximo estágio
- Disponibilizar menu de três pontinhos no card para editar dados do lead
- Exibir indicador visual de temperatura (frio/morno/quente) nos cards

Testes e Qualidade

- Testar fluxo completo de autenticação (login, JWT expirado, logout)
- Testar CRUD de leads integrado ao backend (criar, editar, filtrar, excluir)
- Testar criação e atualização de negociações com validação de regras de negócio
- Testar controle de acesso por perfil (rotas protegidas por role)
- Corrigir e reexecutar testes unitários: leadsAdapter.test.ts e useKanbanBoard.test.ts

Documentação Final (US20)

- Atualizar README com instruções completas de instalação e execução via Docker
- Documentar todos os endpoints da API (Swagger ou README estruturado), incluindo parâmetros temporais
- Incluir DER (Diagrama Entidade-Relacionamento) atualizado
- Incluir Diagrama UML de classes ou de sequência dos fluxos principais
- Adicionar prints finais do sistema no README
- Incluir documento de backlog da Sprint 3
- Preencher retrospectiva da Sprint 2 com conteúdo real
- Descrever padrão arquitetural adotado e padrões GoF utilizados (RP06)

Nesta sprint, o foco é a entrega de um sistema funcional de ponta a ponta, com dados reais, fluxo de negociações completo, controle de acesso e documentação finalizada — consolidando o trabalho desenvolvido nas sprints anteriores.